Vrije Universiteit Amsterdam

Bachelor Thesis - Computer Science

Real-Time Incremental ASR Integration in the Social Interaction Cloud

Author: Daniil Malygin (2763654)

Supervisor: Koen Hindriks
Second reader: Mike Ligthart

A thesis submitted in fulfillment of the requirements for the VU Bachelor of Science degree in Computer Science

July 31, 2025

Abstract

In this project, I propose the integration of a real-time, local, and incremental ASR system into the Social Interaction Cloud (SIC) platform. Currently, SIC uses either Whisper, a batch-processing ASR model, or Dialogflow, a cloud-based NLU service that relies on Google's cloud speech-to-text for audio input. Neither of them provide real-time, incremental speech recognition locally. To address these limitations, this project integrates a lightweight, open-source ASR model into SIC as a dedicated component. After evaluating multiple pre-trained streaming models from the Sherpa-ONNX library on performance metrics such as Real-Time Factor (RTF) and Word Error Rate (WER), the most efficient model was selected and deployed within SIC. The results show that a streaming pre-trained model from Sherpa-ONNX can be successfully embedded into SIC and serve as a potential alternative to Whisper and Dialogflow.

1 Introduction

Social robots are becoming more and more important in fields such as healthcare and education [1]. In those fields, "natural" human-robot interaction is crucial. That means verbal exchanges in which the robot begins to respond almost instantly after a user's turn ending [2]. Achieving such timing requires incremental Automatic Speech Recognition (ASR) [3]. That means using a streaming ASR model. Streaming ASR models process input audio incrementally, which enables partial transcription results during speech. Thus, improving perceived responsiveness.

The Social Interaction Cloud (SIC) is a modular framework designed to support these kinds of applications [4]. It enables flexible configuration of components such as ASR, Text-to-Speech (TTS), and Natural Language Understanding (NLU). These components communicate through service calls or APIs, and are organized into pipelines that process audio, text, or other input/output formats [4]. SIC is designed to be lightweight and flexible. It can run entirely on local devices, and it allows components to be easily swapped or reconfigured to suit different use cases. SIC currently uses Whisper and Dialogflow. Whisper is a high quality, multilingual model, which was released by OpenAI, but it is non-streaming. That means the system waits until a person has finished speaking before it starts processing the batch of words. It provides decent transcription accuracy, but there is some latency involved. Dialogflow is Google's cloud NLU service. It can process some live audio input, but it depends on Google's cloud-based speech-to-text for transcription. That introduces privacy concerns, as well as local capability concerns. Therefore, neither Whisper or Dialogflow provide local, incremental ASR with good processing speed, where partial results are shown during live speech.

This project addresses that limitation by integrating a local, real-time, streaming ASR module into the SIC framework. The integration aims to enable SIC to handle live audio input and produce immediate partial transcriptions.

The main research question is: "How can a local, streaming ASR model be effectively integrated into the Social Interaction Cloud (SIC) to enable real-time, incremental speech recognition?" To explore this, the project investigates three key aspects: researching suitable open-source toolkits and their pre-trained models; testing the performance of those models in terms of decoding speed and transcription accuracy on a powerful desktop, and

integrating the best performing model into the Social Interaction Cloud. Evaluation focuses on two main metrics: Real-Time Factor (RTF), which measures how fast the system transcribes, and Word Error Rate (WER), which measures transcription accuracy.

2 Related Work

2.1 Overview of ASR Systems

Automatic Speech Recognition (ASR) is the technology that converts spoken language into written text. Modern ASR systems can be divided into two main categories: time and deployment. In terms of time, offline batch ASR processes entire utterances or audio files at once, producing a transcription only after the input is fully spoken. On the other hand, real-time streaming ASR analyses audio as it is being spoken. Batch ASR can achieve higher accuracy because of the access to the full context. However, it can do that at the expense of delayed output. Streaming ASR offers low latency with partial results, which is good for interactive applications such as voice assistants or social robots [5], [6].

In terms of deployment, cloud-based ASR delegates speech recognition to remote servers. One example is Google's cloud Speech-to-Text service [7]. It is often integrated into platforms like Dialogflow for voice-based applications. This approach allows to leverage large-scale models and a lot of training data, leading to good accuracy. However, it introduces latency and privacy concerns as it requires a network connection [8]. On the other hand, local ASR runs on a robot or user's device, without internet dependency. That allows to preserve privacy since data never leaves the device. Additionally, local deployment reduces latency, which is crucial for real-time human-robot interaction [9].

ASR systems are usually measured using Word Error Rate (WER). This measures how many mistakes are in the transcription when compared to a given text. Lower WER indicates higher transcription accuracy and is a standard benchmark for comparing ASR models. Usually, anything below 10% is considered very good and means the ASR system is quite accurate [10]. Besides Word Error Rate (WER), ASR systems are evaluated using a variety of other metrics such as for example, the Real-Time Factor (RTF) which measures the decoding speed.

2.2 Overview of Model Architectures

There are a lot of recent improvements in Automatic Speech Recognition (ASR), which have led to the development of powerful neural architectures. Those architectures balance accuracy and computational efficiency. Three main architectures are: Conformer, FastConformer, and Zipformer.

The Conformer architecture is a type of neural network designed for speech recognition that combines convolution and transformers. Convolution helps the model understand short-term patterns in speech, like individual sounds or syllables. Whereas, transformers help it understand the overall meaning of a sentence. This hybrid structure enables high transcription accuracy on benchmarks like LibriSpeech, achieving around 2.1% WER on the `test-clean` dataset [11]. However, it remains computationally intensive and can be slow for real-time applications.

FastConformer is a faster (2.8x times faster) [12], lighter version of the original Conformer. It was created to be more practical for real-time use. It simplifies the transformer part and uses more efficient layers [12]. This makes it much quicker to run, especially on devices with GPUs. Therefore, FastConformer is better suited for streaming speech recognition tasks where low delay is important, without sacrificing too much accuracy.

Zipformer is a newer architecture built specifically for fast, real-time speech recognition [13]. It uses a technique called "zipping," which means it gradually reduces how much audio detail it processes at once. This makes the model smaller and faster without losing too much accuracy. Because of its lightweight design and ability to process speech in real time, Zipformer is ideal for use in streaming ASR.

These differences between the architectures relate to the model selection strategy used later in this thesis.

2.3 Real-Time Incremental ASR Toolkits & Models

In the past couple years, a lot of open-source streaming ASR toolkits and models have been released that support real-time transcription.

Some models are based on the FastConformer architecture [14], an optimized variant of the Conformer model developed by NVIDIA. These models are available through the NeMo toolkit [15], which provides native streaming support with configurable lookahead latencies and is highly optimized for GPU inference using PyTorch. This enables applications to receive partial results as speech is being processed [6]. The architecture of FastConformer minimizes latency and speeds up inference compared to traditional buffered streaming approaches [16].

Sherpa-ONNX is another open source toolkit that is worth mentioning. It includes pre-trained models that are well suited for incremental ASR tasks [17]. One of its key advantages is the support for models exported to the ONNX format. That enables efficient deployment across platforms using ONNX Runtime or TensorRT. It includes a FastConformer model mentioned above, but it offers reduced inference latency while keeping good transcription accuracy [18]. This makes it a more lightweight and deployable alternative to the original NeMo implementation mentioned above. Additionally, Sherpa-ONNX features the Streaming Zipformer, a newer architecture developed by the K2 team. It is explicitly made for efficient, incremental decoding [13]. Zipformer models have been benchmarked on the LibriSpeech test-clean dataset. Reported Word Error Rates (WER) for such models vary depending on model size and training data. For example, the `sherpa-ncnn-streaming-zipformer-20M` model achieves a WER of 3.88% on test-clean [19]. Larger models trained on the full LibriSpeech-960hr dataset achieve WERs as low as 1.78%–2.42% [20] [21]. Also, some of the larger versions (65M–148M parameters) have achieved WERs near 2.0% or below [20] [21]. These results demonstrate that Sherpa-ONNX Zipformer models perform competitively with the state-of-the-art architectures on standard English ASR benchmarks. Therefore, Sherpa-ONNX stands out as a promising toolkit for integration into modular frameworks like the Social Interaction Cloud (SIC).

Besides FastConformer and Sherpa-ONNX models, several other open-source streaming ASR models offer support for real-time transcription and local use. One of those is Vosk, which is a lightweight, open-source speech recognition toolkit [22]. Vosk is engineered for simplicity and performance, requiring minimal computational resources [23]. However,

even though Vosk is a lightweight and efficient ASR toolkit, is not an ideal fit compared to more advanced models like NVIDIA FastConformer and Sherpa-ONNX for integration into the Social Interaction Cloud (SIC). Its transcription accuracy (WER) is significantly lower than that of state-of-the-art architectures such as FastConformer [24]. For example, the default Vosk models typically report WERs around 6–15%, which is noticeably higher than FastConformer or Zipformer models when tested on LibriSpeech datasets [24]. Even though, its simple and efficient, it does not meet the performance standards necessary for a goof ASR system to be integrated into the Social Interaction Cloud(SIC).

Another one is Faster-Whisper, which is a lightweight, Python-native implementation of OpenAI's Whisper ASR model. It can transcribe audio up to 4x faster than the original OpenAI Whisper implementation, while also maintaining the same level of accuracy [25]. It maintains WER of around 2.7% on test-clean sets, which matches the original model's performance [26]. Despite being potentially easy to integrate into SIC due to an already existing Whisper implementation, Faster-Whisper is not a suitable candidate for this thesis project because it does not support true streaming or incremental speech recognition [27]. For a truly real-time system, models like Sherpa-ONNX or FastConformer are better suited, as they provide genuine incremental decoding capabilities.

SpeechBrain is yet another open-source conversational AI toolkit built on PyTorch. The toolkit comes with a wide selection of pre-trained models. Its models are competitive in terms of accuracy and can be optimized for speed and efficiency [28]. A promising candidate is the `speechbrain/asr-streaming-conformer-librispeech` model. It is developed within the SpeechBrain framework [29]. It supports real-time streaming through dynamic chunk training, which allows it to process audio in configurable windows ranging from 80 ms to 1280 ms [30]. This makes it suitable for latency-sensitive applications. The model offers real-time incremental transcription output. Its integration-friendly Python API, which supports both GPU and CPU inference, makes it a promising option for local deployment in SIC. [30].

Certain ASR solutions, such as LLaMA-Omni, were excluded from this study due to their end-to-end design and limited modularity [31]. While such models often offer strong performance, they typically lack the flexibility needed for integration into a modular framework like SIC. In particular, their architecture makes it difficult to isolate or replace specific components [31]. Therefore, since SIC is built for replacement of individual components, models like Omni do not align with the project's goal.

Based on analysis above, three toolkits/models stood out as the most promising for further study. Those were: NVIDIA NeMo FastConformer; Sherpa-ONNX Zipformer and SpeechBrain Streaming Conformer. They offer the best balance of low-latency incremental decoding, good WER, active open-source support, as well as local deployment on GPUs.

3 Methodology

3.1 Choosing a Single Toolkit: Sherpa-ONNX

After reviewing various toolkits and pretrained models in the "Related Work" section, it was decided to work exclusively with the Sherpa-ONNX toolkit and evaluate three pretrained streaming models from it. This decision offers several advantages. First, Sherpa-ONNX includes support for both the FastConformer (originally from NVIDIA NeMo) and

its own Zipformer architecture, covering two of the top-performing candidates identified earlier. Second, Sherpa-ONNX offers a lightweight interface, making it significantly easier to test multiple models using the same codebase. All three selected models can be evaluated by simply changing the model directory within a single script. Therefore, it significantly reduces development and debugging time.

3.2 Selecting Three Models

Three pretrained streaming ASR models were selected from the Sherpa-ONNX model library based on their promising speed and Word Error Rate (WER) performance, as well as architectural diversity. The goal was to compare models that offer different trade-offs between speed and accuracy.

The selected models were:

1. `sherpa-onnx-nemo-streaming-fast-conformer-transducer-en-80ms` — a FastConformer-based model originally developed by NVIDIA NeMo, optimized for fast decoding;

2. `sherpa-onnx-streaming-zipformer-en-20M-2023-02-17` — a lightweight Zipformer model with fewer parameters, designed for use on edge devices;

3. `sherpa-onnx-streaming-zipformer-en-2023-06-21` — a more recent Zipformer variant offering improved accuracy and streaming stability.

3.3 Evaluation Approach

To benchmark the performance, there was a two-stage evaluation: (i) tests on the small set of 3 WAV files provided within the model directories, and (ii) tests on the standardized LibriSpeech test-cleat dataset to assess the models more accurately. All evaluations were performed on the same hardware. The evaluation focused on two key performance metrics: decoding speed, measured by Real-Time Factor (RTF), and transcription accuracy, measured by Word Error Rate (WER).

3.3.1 Desktop Benchmark

Before actual benchmarking mentioned in the evaluation approach, each model was first tested in a minimal script to ensure it loaded correctly and produced transcription on live speech. This early check confirmed that the models were functioning properly after being loaded. After this initial validation, a comparative evaluation was done on a desktop machine to identify the most suitable ASR model for integration into SIC.

During the 3 WAV Files Test, in order to test the models in their most optimal configurations, I experimented with the number of decoding threads (`num_threads`). Each model was evaluated using 2, 4, 6, and 8 threads to observe how decoding speed was affected. Each model was ran 3 times per each thread count, so in total 12 times. This was done to compute the average RTF under each configuration and select the optimal thread count for each model. After determining the best thread count, each model was tested once on the full LibriSpeech `test-clean` dataset. Running this test multiple times was avoided because of its long execution time (approximately 30 minutes per model). More information on the results of varying of the threads is mentioned in the "Results" section. The

model that demonstrated the best trade-off between good RTF and transcription accuracy was selected for integration into the SIC framework.

3.4 SIC Integration Approach

To integrate the selected ASR model into the Social Interaction Cloud (SIC), a standard structure was followed. There was one component script acting as a service, and one demo script acting as an application. The component was placed in the `social-interaction-cloud` repository. The demo script was developed within the `sic_applications` repository. Both repositories were cloned into a single development folder named `sic_dev`. Following the recommended SIC architecture, the system was run across three separate terminals: one for the Redis server, one for the service script, and one for the demo script.

4 Design & Implementation

4.1 Hardware Specs

Everything was done on a local machine running Ubuntu 20.04.6 LTS. The system was equipped with an **Intel Core i7-12650H** CPU (12th Gen, 10 cores, 16 threads) and an **NVIDIA GeForce RTX 4070 Laptop GPU** with 8 GB of VRAM. This hardware allowed for GPU-accelerated testing using ONNX Runtime with CUDA backend.

Hardware information was collected using tools such as `lscpu` for CPU specifications and `nvidia-smi` for GPU and CUDA runtime information.

4.2 Desktop Setup

This section describes the technical setup and implementation details for running the selected ASR models outside of the SIC framework. These desktop tests served as the basis for benchmarking and the selection of the best model. It allowed for controlled evaluation before integration into SIC. The following subsections outline the working environment & dependency specifications, test configurations, and various challenges encountered during this phase. Everything was done inside a project folder named `sherpa-onnx-demo`.

4.2.1 Environment & Dependencies

Setting up the local development environment was one of the toughest parts of this project. Since the benchmarks were GPU-accelerated, proper installation of CUDA and matching Python-compatible packages was essential to ensure compatibility with Sherpa-ONNX and ONNX Runtime.

Initial system verification was performed using `nvidia-smi`, which confirmed the presence of the GPU and the necessary driver (535.183.01), as well as the supported CUDA version (12.2 and below).

Although the installed NVIDIA driver supported CUDA version 12.2, CUDA 11.8 was selected instead to ensure compatibility with the Python 3.10 environment used in this project. Python 3.10 was chosen for its broad support across key libraries such as Sherpa-ONNX and PyTorch, both of which offer stable CUDA 11.8 wheels optimized for GPU acceleration.

CUDA 11.8 was manually installed into a directory (`$HOME/cuda-11.8`) using the official NVIDIA `.run` installer [32], rather than the system-wide default path. This approach was necessary because, at the time of setup, administrative (sudo) privileges were not yet available to me on the machine. As a result, I manually added the runtime paths to the shell configuration file to make the CUDA toolkit accessible. Although system-wide installation via `apt` or to `/usr/local/cuda` became possible later after obtaining sudo access, the local setup was already functional and I kept using it throughout the project.

The following environment variables were added to the shell configuration file (`~/.bashrc`) to expose the correct CUDA paths to the system:

```
export PATH=$HOME/cuda-11.8/bin:$PATH
export LD_LIBRARY_PATH=$HOME/cuda-11.8/lib64:$LD_LIBRARY_PATH
```

After sourcing the updated shell configuration, the required dependencies were installed via `pip`:

- `torch`, `torchvision`, and `torchaudio` (with CUDA 11.8 support) from the official PyTorch CUDA wheel index [33]:

  ```
  pip install torch torchvision torchaudio
  --index-url https://download.pytorch.org/whl/cu118
  ```

- `sherpa-onnx` (version 1.11.1+cuda) from the Sherpa-ONNX CUDA wheel repository [34]:

  ```
  pip install sherpa-onnx==1.11.1+cuda -f
  https://k2-fsa.github.io/sherpa/onnx/cuda.html
  ```

- `onnxruntime-gpu` (version 1.20.1) [35]:

  ```
  pip install onnxruntime-gpu==1.20.1
  ```

The Sherpa-ONNX installation used the `+cuda` variant to ensure GPU support. All packages and runtime configurations were selected to align with the Python 3.10 interpreter and the CUDA 11.8 environment [32].

Other Python packages were installed, because they were needed to support audio handling, and model execution. Some of these were `sounddevice` [36] and `soundfile` [37], which were required for audio input/output, while others like `jiwer` [38] and `RapidFuzz` were used for computing evaluation metrics such as Word Error Rate (WER). Several dependencies, including `numpy`, `protobuf`, and `flatbuffers`, were installed to satisfy Sherpa-ONNX and ONNX Runtime requirements.

Finally, the pretrained ASR models were downloaded directly from the official Sherpa-ONNX release repository [39]. For example, one of the selected models was installed as follows:

```
wget https://github.com/k2-fsa/sherpa-onnx/releases/download/asr-models/
sherpa-onnx-streaming-zipformer-en-2023-06-21.tar.bz2

tar -xvf sherpa-onnx-streaming-zipformer-en-2023-06-21.tar.bz2
```

The other two models were installed in the same way. The next section explains the vanilla setup that was done to ensure the models were installed properly and the environment & dependencies were set up properly as well.

4.2.2 Vanilla Test Design

Before any structured benchmarking was done, a lightweight test script (`app.py`) was created to verify that the selected ASR models could load correctly and perform real-time transcription. The script uses the Sherpa-ONNX Python API to capture live audio input from the microphone, run streaming inference in small chunks, and display transcriptions in the terminal [40]. A single variable at the top of the script defines the model directory, allowing easy switching between the three candidates by commenting or uncommenting one line of code.

This test helped to ensure the models were functioning as expected before proceeding with the 3 WAV Files and LibriSpeech tests. When running the script, the `watch -n 0.5 nvidia-smi` command was used in a separate terminal window to confirm that GPU was active while the model processed live audio input. A full version of the script is included in Appendix A.1.

4.2.3 3 WAV Files Test Design

To compare the performance of all three ASR models on the pre-included 3 WAV Files, a (`wer_rtf.py`) script was created. These files come with corresponding "ground truth" transcriptions. The script measures both transcription accuracy and decoding efficiency by computing Word Error Rate (WER) and Real-Time Factor (RTF).

Each WAV file is first fully loaded into memory. However, to test how the models behave in streaming conditions, the audio is not passed to the recognizer in a single block. Instead, it is "sliced" into fixed-duration chunks of 20 milliseconds, determined by the `FRAME_DUR` constant. These chunks are then incrementally fed into Sherpa-ONNX's `Online Recognizer` via `accept_waveform()`, and decoding is performed only when the recognizer is ready using `is_ready()`.

For each input file, the script tracks both total audio duration and cumulative decoding time. These metrics are used to compute the RTF, which indicates how efficiently the model processes input audio in real-time. The final transcription hypothesis is compared against the reference transcript using the `jiwer` package to calculate the Word Error Rate (WER). A full version of the script is included in Appendix A.2.

Models were tested with various thread counts, which was done by changing the `num_threads` variable.

4.2.4 LibriSpeech Test Design

To complement the 3 WAV evaluation and provide a more meaningful performance comparison, a second benchmarking script `wer_rtf_librispeech.py` was developed to evalu-

ate the selected models on the `test-clean` subset of the LibriSpeech dataset [41]. This subset consists of approximately 5 hours of speech across 2,600 utterances and is widely used as a standard benchmark in ASR research. This was done to potentially see more stable and comparable results across models. It was downloaded by running the following commands:

```
wget https://www.openslr.org/resources/12/test-clean.tar.gz
tar -xvzf test-clean.tar.gz
```

The test begins by scanning the dataset to collect all audio files and their reference transcriptions. Each utterance is then loaded and fed into the Sherpa-ONNX recognizer in fixed-size chunks (20 ms), similar to the previous test. The script again tracks the total duration of audio processed and the decoding time, which allows for the computation of the RTF and WER.

Unlike the 3 WAV test, this test ensures that performance differences are not due to small sample bias. All models were tested using their optimal thread counts as determined in the first benchmark. A full version of the script is included in Appendix A.3.

4.2.5 Challenges

Several challenges were encountered during the desktop setup and environment configuration phase. The most time-consuming issues were related to selecting the correct Python version and matching it with the appropriate CUDA and Sherpa-ONNX wheels. Some pip packages had to be installed based on missing dependency errors during runtime. One particularly frustrating issue involved the GPU not being utilized, and the machine falling back on CPU, even though all packages appeared to be correctly installed. After investigating, it turned out the problem was caused by an incorrectly configured `LD_LIBRARY_PATH`, which prevented the system from accessing the CUDA runtime libraries. Once the correct environment variables were exported, GPU acceleration functioned as expected.

4.3 SIC Setup

After identifying the best-performing ASR model from the desktop benchmarking stage, the next step was to integrate it into the Social Interaction Cloud (SIC) framework. The selected model, `sherpa-onnx-streaming-zipformer-en- 2023-06-21`, was chosen based on its trade-off between speed and transcription accuracy.

To prepare for this integration, I first familiarized myself with the structure of the SIC framework by exploring its official tutorials and example projects available on GitHub [42]. I then created a local development workspace (`sic_dev`) and cloned the necessary repositories: `social-interaction-cloud` [43] and `sic_applications` [44].

The Python environment used for running SIC was created as a separate virtual environment (`venv_sic`) but followed the same configuration as the desktop setup. It included the same Python version, CUDA version, and Sherpa-ONNX installation. Also, to support local development and allow dynamic imports across components, the base SIC repository was also installed in editable mode using `pip install -e .`, and a `.env` file was used to set the appropriate `PYTHONPATH`. This ensured that model performance could be evaluated under consistent technical conditions. This setup provided a modular environment in which I could implement my ASR component.

4.3.1 Service Script (ASR Component)

To integrate the selected ASR model into SIC as a microservice, a new component was created under the `social-interaction-cloud` repository at `sic_framework/services/sherpa_asr`. This component had two files: `sherpa_service.py`, which implemented the actual logic of the ASR service, and an `__init__.py` file for module-level imports. The core of the component was a wrapper class around the Sherpa-ONNX recognizer, which handled initialization, streaming state, and decoding in a background thread. Audio messages were received via the standard SIC interface and queued for asynchronous processing. The decoding loop consumed audio chunks and produced partial transcripts whenever new output became available. If the incoming sample rate differed from 16 kHz, the signal was resampled using `scipy` to meet model requirements. Communication between components followed SIC's message-based architecture, using `SICMessage` and `SICRequest` classes to trigger inference or retrieve results. A complete version of the script is available in Appendix A.4.

4.3.2 Demo Script

To complete the integration of my component within SIC, a lightweight demo script was implemented and placed at `sic_applications/demos/`.

`desktop/demo_desktop_microphone_sherpa.py`. This script acts as a client application that initializes the SIC desktop microphone, connects it to the `SICSherpa` component, and registers a callback to receive and print transcription updates. Once launched, the script begins listening to microphone input and continuously routes audio to the ASR component. When a new transcript is published by the component, it is received through the `on_transcript` function and printed to the terminal in real time. The loop runs until the user manually exits the application. This script followed the same message-passing architecture as other SIC demos [45]. A complete demo script is included in Appendix A.5.

4.3.3 Running the Demo & Tests

To run any of the ASR experiments within the SIC framework, a consistent three-terminal setup was used to launch and coordinate everything. This structure follows the standard architecture of SIC, where everything communicates via a central Redis server. The following steps were used:

- **Terminal 1 — Start the Redis server:**
  `redis-server conf/redis/redis.conf`
- **Terminal 2 — Launch the Sherpa ASR component:**
  `python3 sherpa_service.py`
- **Terminal 3 — Run the demo script:**
  - To run the microphone demo:
    `python3 demo_desktop_microphone_sherpa.py`

It is important to note that each script must be executed from the correct working directory to ensure that relative imports and file paths resolve properly. For example, the

ASR service script should run from within the `/sic_dev/social-interaction-cloud/sic_framework/services/sherpa_asr` folder, while the demo script must be launched from it's respective directory. Failing to do so may result in import errors or incorrect path resolution for the model files.

4.3.4 Challenges

One recurring issue was an import warning when attempting to run the demo script . This was resolved by explicitly setting the `PYTHONPATH` to include the SIC root directory. A `.env` file was created with the following line:

```
PYTHONPATH=/home/daniil/sic_dev/social-interaction-cloud
```

This ensured that local component modules could be imported correctly when launching scripts from nested folders.

5 Results

This section presents the results of the two-stage evaluation process.

5.1 Desktop 3 WAV Files Test Results

5.1.1 Model 1

Table 1: Average WER and RTF for `sherpa-onnx-streaming-zipformer-en-20M-2023-02-17` across different thread counts.

Threads	Average WER	Average RTF
2	0.14285	0.02086
4	0.14285	0.02340
6	0.14285	0.03475
8	0.14285	0.03507

5.1.2 Model 2

Table 2: Average WER and RTF for `sherpa-onnx-nemo-streaming-fast-conformer-transducer-en-80ms` across different thread counts.

Threads	Average WER	Average RTF
2	0.14285	0.16990
4	0.14285	0.14806
6	0.14285	0.21530
8	0.14285	0.41132

5.1.3 Model 3

Table 3: Average WER and RTF for `sherpa-onnx-streaming-zipformer-en-2023-06-21` across different thread counts.

Threads	Average WER	Average RTF
2	0.03896	0.07654
4	0.03896	0.07631
6	0.03896	0.07499
8	0.03896	0.08135

5.1.4 Comparison on 3 WAV Files & Optimal Threads

Table 4: Optimal thread configuration for each model based on lowest average RTF and stable WER.

Model	Optimal Threads	WER	RTF
`Model 1`	2	0.14285	0.02086
`Model 2`	4	0.14285	0.14806
`Model 3`	6	0.03896	0.07499

5.2 Desktop LibriSpeech Test Results

5.2.1 Model 1

Table 5: LibriSpeech test-clean results for `sherpa-onnx-streaming-zipformer-en-20M-2023-02-17` with 2 threads.

Total Utterances	WER	RTF
2620	0.2577	0.0325

5.2.2 Model 2

Table 6: LibriSpeech test-clean results for `sherpa-onnx-nemo-streaming-fast-conformer-transducer-en-80ms` with 4 threads.

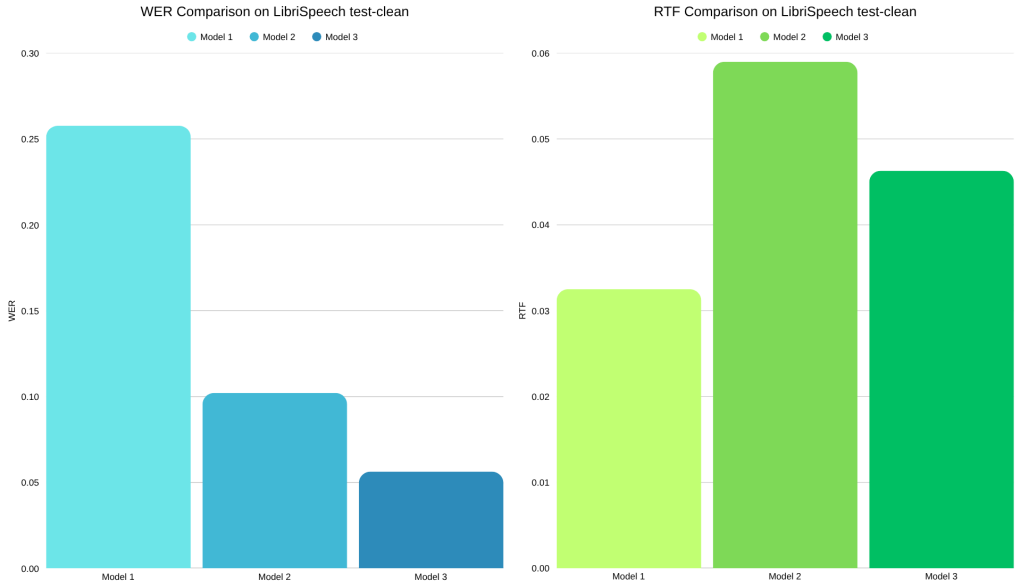
Total Utterances	WER	RTF
2620	0.1021	0.0590

5.2.3 Model 3

Table 7: LibriSpeech test-clean results for `sherpa-onnx-streaming-zipformer-en-2023-06-21` with 6 threads.

Total Utterances	WER	RTF
2620	0.0563	0.0463

5.2.4 Comparison on LibriSpeech test-clean



6 Discussion

6.1 Desktop 3 WAV Files & Optimal Threads

Across all three models and all tested thread configurations, the Word Error Rate (WER) remained identical for each run. This consistency showed that WER was independent of the number of decoding threads did not influence transcription accuracy.

In contrast, the Real-Time Factor (RTF) was significantly impacted by the number of threads. As the number of threads increased, some models exhibited improved performance up to a point (e.g., lower RTFs), while others showed diminishing returns or even regression. For example, Model 1 achieved its lowest RTF at 2 threads (RTF = 0.02086), while Model 3 performed best at 6 threads (RTF = 0.07499). Model 2, on the other hand, showed an increase in RTF at 8 threads, suggesting that more threads may introduce overhead or synchronization issues depending on the model's internal architecture or implementation.

This demonstrates that optimal thread configuration must be chosen carefully, as it directly affects speed. It is not safe to assume that more threads always lead to better performance. Instead, minimal benchmarking like the 3 WAV test used here is valuable for determining the thread setting that yields the best balance between efficiency and

computational cost. Table 4 summarizes the optimal configuration for each model based on lowest RTF.

6.2 Desktop LibriSpeech

After identifying the optimal thread count for each model through the 3 WAV files benchmark, a more comprehensive evaluation was performed using the 5-hour LibriSpeech test-clean dataset. Each model was evaluated using its optimal configuration: Model 1 with 2 threads, Model 2 with 4 threads, and Model 3 with 6 threads.

As shown in Tables 5–7 and visualized in Figure 5.2.4, Model 3 once again outperformed the others with the lowest WER of 0.0563 and achieved an RTF of 0.0463. Model 2 followed with a WER of 0.1021 and RTF of 0.0590, while Model 1 had the highest WER of 0.2577 but the lowest RTF of 0.0325. These results confirm the findings from the smaller benchmark and highlight the superior accuracy of Model 3, despite slightly higher decoding time than Model 1. Therefore, Model 3 was the winner.

Although the Word Error Rates (WER) obtained in this thesis are higher than those reported by the developers of Sherpa-ONNX on the same LibriSpeech test-clean dataset, such results are expected due to differences in benchmarking conditions (e.g hardware or modified decoding). Despite this, Model 3 (sherpa-onnx-streaming-zipformer-en-2023-06-21) demonstrated strong relative performance and consistent superiority over the other candidates.

The fact that Model 3 achieved an RTF of (0.0463), which is less than 1.0, means it processes audio more than 20 times faster than real-time, confirming its suitability for real-time streaming applications.

6.3 Integration into SIC

The component operated reliably in real-time, processing audio input incrementally and publishing transcription results within the expected communication pipeline. The integration followed the standard SIC development practices, using multiple terminals. The service responded correctly to both continuous audio input and transcript requests, which confirms compatibility with SIC's input/output message structure. Overall, the integration effort validates the thesis claim: it is possible to embed and run a real-time incremental ASR model inside the Social Interaction Cloud.

7 Conclusion & Future Work

This thesis set out to determine whether a real-time, incremental Automatic Speech Recognition (ASR) model could be successfully integrated into the Social Interaction Cloud (SIC). The findings confirm that this is indeed feasible. `sherpa-onnx-streaming-zipformer-en-2023-06-21`, was selected based on benchmarking results and was fully integrated into SIC as a functional microservice.

However, although this thesis does not include formal benchmarking results inside the Social Interaction Cloud (SIC), preliminary attempts were made to integrate and test the selected ASR model within the SIC architecture. These tests revealed inflated WER

and RTF values due to architectural factors such as delays and the lack of explicit decoding flushing. Since these issues introduced confusing results and the benchmarking environment was no longer accessible for re-testing, the results were excluded. Future work should include rigorous SIC benchmarking to validate model performance in the full system context.

Also, to determine whether the new model should replace the current Whisper or Dialogflow implementation, a direct comparison is needed. Testing them under identical conditions inside SIC and on physical robots would provide clearer insight into the model's real-world usability. Until such a comparison is conducted, it is too early to make a definitive recommendation for switching to the `sherpa-onnx-streaming-zipformer-en-2023-06-21` model.

Future work should focus on several key areas. First, benchmarking the model under the SIC conditions. Second, integrating the model with downstream services & components such as Natural Language Understanding (NLU) modules and LLMs, which would enable a full pipeline and allow for more complex use cases. Third, testing the integrated model directly on robots such as Pepper in human-robot interaction contexts. Finally, experimenting with other open-source real-time incremental models could help identify architectures that introduce less overhead and scale better within multi-component systems like the Social Interaction Cloud (SIC) then the model that was chosen for this thesis.

In conclusion, this project demonstrates that a real-time ASR integration into SIC is achievable and opens the door to further research.

References

[1] T. Belpaeme, J. Kennedy, A. Ramachandran, B. Scassellati, and F. Tanaka, "Social robots for education: A review," *Science Robotics*, vol. 3, no. 21, p. eaat5954, 2018.

[2] K. Pitsch, H. Kuzuoka, Y. Suzuki, and K. Yamazaki, "Timing and responsiveness in human-robot interaction," *Interaction Studies*, vol. 14, no. 3, pp. 455–486, 2013.

[3] G. Skantze and D. Schlangen, "Exploring the impact of feedback and incremental speech recognition in a human-machine dialogue system," *Computer Speech & Language*, vol. 25, no. 1, pp. 117–137, 2011.

[4] U. o. T. Social AI Group, "Social interaction cloud (sic) framework," https://socialrobotics.atlassian.net/wiki/spaces/SIC/overview, 2023, accessed: 2025-07-28.

[5] J. Yu, W. Han, A. Gulati, C.-C. Chiu, B. Li, T. N. Sainath, Y. Wu, and R. Pang, "Dual-mode {asr}: Unify and improve streaming {asr} with full-context modeling," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=Pz_dcqfcKW8

[6] V. Noroozi, S. Majumdar, A. Kumar, J. Balam, and B. Ginsburg, "Stateful conformer with cache-based inference for streaming automatic speech recognition," in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and*

Signal Processing (ICASSP). IEEE, Apr. 2024, p. 12041–12045. [Online]. Available: http://dx.doi.org/10.1109/ICASSP48485.2024.10446861

[7] Google Cloud, "Speech-to-text documentation," https://cloud.google.com/speech-to-text, 2024, accessed: 2025-07-28.

[8] L. Deng and G. Hinton, "Deep learning for speech recognition: Challenges and opportunities," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, 2012, discusses trade-offs between cloud and on-device ASR.

[9] A. S. Sharma and R. Bhalley, "Asr — a real-time speech recognition on portable devices," in *2016 2nd International Conference on Advances in Computing, Communication, & Automation (ICACCA) (Fall)*. IEEE, Sep. 2016, p. 1–4. [Online]. Available: http://dx.doi.org/10.1109/ICACCAF.2016.7749004

[10] A. Ali and S. Renals, "Word error rate estimation for speech recognition: e-wer," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, 2018. [Online]. Available: http://dx.doi.org/10.18653/v1/P18-2004

[11] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and R. Pang, "Conformer: Convolution-augmented transformer for speech recognition," 2020. [Online]. Available: https://arxiv.org/abs/2005.08100

[12] D. Rekesh, N. R. Koluguri, S. Kriman, S. Majumdar, V. Noroozi, H. Huang, O. Hrinchuk, K. Puvvada, A. Kumar, J. Balam, and B. Ginsburg, "Fast conformer with linearly scalable attention for efficient speech recognition," 2023. [Online]. Available: https://arxiv.org/abs/2305.05084

[13] Z. Yao, L. Guo, X. Yang, W. Kang, F. Kuang, Y. Yang, Z. Jin, L. Lin, and D. Povey, "Zipformer: A faster and better encoder for automatic speech recognition," 2023. [Online]. Available: https://arxiv.org/abs/2310.11230

[14] NVIDIA, "Nvidia fastconformer hybrid large (streaming, multi file)," https://huggingface.co/nvidia/stt_en_fastconformer_hybrid_large_streaming_multi, 2024, accessed: 2025-07-30.

[15] ——, "Nemo asr models - fastconformer," https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/asr/models.html#fast-conformer, 2024, accessed: 2025-07-30.

[16] M. Xu, A. Jin, S. Wang, M. Su, T. Ng, H. Mason, S. Han, Z. Lei, Y. Deng, Z. Huang, and M. Krishnamoorthy, "Conformer-based speech recognition on extreme edge-computing devices," 2023. [Online]. Available: https://arxiv.org/abs/2312.10359

[17] K. Community, "Sherpa-onnx: Streaming and offline asr with onnx models," https://github.com/k2-fsa/sherpa-onnx, 2024, gitHub repository.

[18] K2-FSA, "Sherpa-onnx pretrained asr models," https://github.com/k2-fsa/sherpa-onnx/releases/tag/asr-models, 2024, accessed: July 2025.

[19] "sherpa-ncnn-streaming-zipformer-20m-2023-02-17 (english) model," Hugging Face model card, 2023, wER reported: 3.88% on LibriSpeech test-clean.

[20] K. F. Team, "Icefall: Speech recognition toolkit based on pytorch and k2," https://github.com/k2-fsa/icefall, 2024, accessed: 2025-07-28.

[21] ——, "Sherpa-ncnn pretrained zipformer transducer models," https://k2-fsa.github.io/sherpa/ncnn/pretrained_models/zipformer-transucer-models.html, 2024, accessed: 2025-07-28.

[22] A. A. Soni, "Improving speech recognition accuracy using custom language models with the vosk toolkit," 2025. [Online]. Available: https://arxiv.org/abs/2503.21025

[23] A. Bratterud, A.-A. Walla, H. Haugerud, P. E. Engelstad, and K. Begnum, "Includeos: A minimal, resource efficient unikernel for cloud services," in *2015 IEEE 7th International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, Nov. 2015, p. 250–257. [Online]. Available: http://dx.doi.org/10.1109/CloudCom.2015.89

[24] AlphaCephei, "Vosk speech recognition toolkit," 2024, available at https://github.com/alphacep/vosk-api.

[25] SYSTRAN, "faster-whisper: Faster whisper transcription with ctranslate2," https://github.com/SYSTRAN/faster-whisper, 2025, gitHub repository.

[26] S. community, "Discussion 1 on systran/faster-whisper-large-v3: ẅhat does faster mean and what advantage this have over original whisper?;," https://huggingface.co/Systran/faster-whisper-large-v3/discussions/1, Dec. 2023, hugging Face discussion; accessed July 28, 2025.

[27] J. W. Kim, "Github discussion: "possible to use for real-time / streaming tasks?"," https://github.com/openai/whisper/discussions/2, Sep. 2022, openAI Whisper GitHub repository discussion; accessed July 28, 2025.

[28] S. Team, "speechbrain/asr-streaming-conformer-librispeech," https://huggingface.co/speechbrain/asr-streaming-conformer-librispeech, 2024, accessed: 2025-07-20.

[29] S. Contributors, "Speechbrain: A pytorch toolkit for speech processing," https://github.com/speechbrain/speechbrain, 2024, accessed: 2025-07-20.

[30] SpeechBrain, "Speechbrain documentation," https://speechbrain.readthedocs.io, 2024, accessed: 2025-07-20.

[31] Q. Fang, S. Guo, Y. Zhou, Z. Ma, S. Zhang, and Y. Feng, "Llama-omni: Seamless speech interaction with large language models," 2024. [Online]. Available: https://arxiv.org/abs/2409.06666

[32] NVIDIA, "Cuda toolkit 11.8 downloads," https://developer.nvidia.com/cuda-11-8-0-download-archive, 2023, accessed: 2025-07-20.

[33] PyTorch Team, "Installing pytorch with cuda 11.8," https://pytorch.org/get-started/locally/, 2023, accessed: 2025-07-20.

[34] K2-FSA, "Sherpa-onnx cuda installation guide," https://k2-fsa.github.io/sherpa/onnx/cuda.html, 2024, accessed: 2025-07-20.

[35] O. R. Team, "Installing onnx runtime gpu," https://onnxruntime.ai/docs/build/eps.html#cuda, 2023, accessed: 2025-07-20.

[36] M. Geier, "sounddevice: Play and record audio in python," https://python-sounddevice.readthedocs.io/, 2024, accessed: 2025-07-20.

[37] B. Bechtold, "soundfile: An audio library based on libsndfile," https://pysoundfile.readthedocs.io/, 2024, accessed: 2025-07-20.

[38] J. Wer, "jiwer: Evaluation measures for speech recognition," https://github.com/jitsi/jiwer, 2024, accessed: 2025-07-20.

[39] K2-FSA, "Sherpa-onnx pretrained asr models," https://github.com/k2-fsa/sherpa-onnx/releases/tag/asr-models, 2024, accessed: 2025-07-20.

[40] ——, "Sherpa-onnx: Python example — speech recognition from microphone," https://github.com/k2-fsa/sherpa-onnx/blob/master/python-api-examples/speech-recognition-from-microphone.py, 2024, accessed: 2025-07-20.

[41] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "LibriSpeech: an ASR corpus based on public domain audio books," in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2015, pp. 5206–5210.

[42] S. R. Lab, "Getting started with the social interaction cloud," 2023, accessed: 2025-07-20. [Online]. Available: https://socialrobotics.atlassian.net/wiki/spaces/CBSR/pages/2180415493/Getting+started

[43] Social AI Group, Vrije Universiteit Amsterdam, "Social interaction cloud (sic)," https://github.com/Social-AI-VU/social-interaction-cloud, 2024, accessed: 2025-07-20.

[44] ——, "Sic applications," https://github.com/Social-AI-VU/sic_applications, 2024, accessed: 2025-07-20.

[45] ——, "Sic applications: Whisper microphone demo script," https://github.com/Social-AI-VU/sic_applications/blob/main/demos/desktop/demo_desktop_microphone_whisper.py, 2024, accessed: 2025-07-20.

A Appendix

A.1 app.py Script

```
import sounddevice as sd
import sherpa_onnx
from pathlib import Path
```

```
MODEL_DIR = "sherpa-onnx-streaming-zipformer-en-2023-06-21"
#MODEL_DIR = "sherpa-onnx-streaming-zipformer-en-20M-2023-02-17"
#MODEL_DIR = "sherpa-onnx-nemo-streaming-fast-conformer-transducer-en-80ms"

def assert_file_exists(x):
    assert Path(x).is_file(), f"{x} not found"

def create_recognizer():
    tokens  = f"{MODEL_DIR}/tokens.txt"
    encoder = f"{MODEL_DIR}/encoder.onnx"
    decoder = f"{MODEL_DIR}/decoder.onnx"
    joiner  = f"{MODEL_DIR}/joiner.onnx"

    for f in (tokens, encoder, decoder, joiner):
        assert_file_exists(f)

    return sherpa_onnx.OnlineRecognizer.from_transducer(
        tokens=tokens,
        encoder=encoder,
        decoder=decoder,
        joiner=joiner,
        num_threads=6,
        provider="cuda",
    )

def main():
    recognizer = create_recognizer()
    stream = recognizer.create_stream()
    sr_in = 16000
    frame_size = int(0.02 * sr_in)

    print("Listening... (Ctrl-C to quit)")

    with sd.InputStream(channels=1, dtype="float32", samplerate=sr_in) as mic:
        last = ""
        while True:
            samples, _ = mic.read(frame_size)
            stream.accept_waveform(sr_in, samples.reshape(-1))
            while recognizer.is_ready(stream):
                recognizer.decode_stream(stream)
            text = recognizer.get_result(stream)
            if text != last:
                print("\r" + text, end="", flush=True)
                last = text
```

```
if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\nDone.")
```

A.2 wer_rtf.py Script

```
import time
from pathlib import Path
import soundfile as sf
import numpy as np
import sherpa_onnx
from jiwer import wer

MODEL_DIR = "sherpa-onnx-streaming-zipformer-en-2023-06-21"
# MODEL_DIR = "sherpa-onnx-streaming-zipformer-en-20M-2023-02-17"
# MODEL_DIR = "sherpa-onnx-nemo-streaming-fast-conformer-transducer-en-80ms"
FRAME_DUR   = 0.02

def load_refs(txt_path):
    refs = {}
    for line in Path(txt_path).read_text().splitlines():
        utt, *rest = line.split(maxsplit=1)
        refs[utt] = rest[0] if rest else ""
    return refs

def main():
    model_dir = Path(__file__).parent / MODEL_DIR
    wav_dir   = model_dir / "test_wavs"
    refs      = load_refs(wav_dir / "trans.txt")

    recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
        tokens      = str(model_dir / "tokens.txt"),
        encoder     = str(model_dir / "encoder.onnx"),
        decoder     = str(model_dir / "decoder.onnx"),
        joiner      = str(model_dir / "joiner.onnx"),
        num_threads = 6, #change for each model, so the optimal is used.
        provider = "cuda",
    )

    total_audio, total_decode = 0.0, 0.0
    hyps = []

    for wav in sorted(wav_dir.glob("*.wav")):
        audio, sr = sf.read(wav, dtype="float32")
```

```
        frame_size = int(sr * FRAME_DUR)
        stream = recognizer.create_stream()
        decode_time = 0.0

        for offset in range(0, len(audio), frame_size):
            chunk = audio[offset : offset + frame_size]
            stream.accept_waveform(sr, chunk)
            while recognizer.is_ready(stream):
                t0 = time.perf_counter()
                recognizer.decode_stream(stream)
                t1 = time.perf_counter()
                decode_time += (t1 - t0)

        while recognizer.is_ready(stream):
            t0 = time.perf_counter()
            recognizer.decode_stream(stream)
            t1 = time.perf_counter()
            decode_time += (t1 - t0)

        hyp = recognizer.get_result(stream)
        hyps.append(hyp)

        audio_len = len(audio) / sr
        total_audio += audio_len
        total_decode += decode_time

    print("=== Summary ===")
    print("WER :", wer(list(refs.values()), hyps))
    print("RTF :", total_decode / total_audio)

if __name__ == "__main__":
    main()
```

A.3 wer_rtf_librispeech.py Script

```
import time
from pathlib import Path
from typing import Dict, List, Tuple
import soundfile as sf
import sherpa_onnx
from jiwer import wer
from tqdm import tqdm

MODEL_DIR = "sherpa-onnx-streaming-zipformer-en-2023-06-21"
# MODEL_DIR = "sherpa-onnx-streaming-zipformer-en-20M-2023-02-17"
# MODEL_DIR = "sherpa-onnx-nemo-streaming-fast-conformer-transducer-en-80ms"
```

```
FRAME_DUR  = 0.02

def scan_librispeech(root: Path) -> Tuple[List[Path], Dict[str, str]]:
    wavs: List[Path] = []
    refs: Dict[str, str] = {}
    for txt in root.rglob("*.trans.txt"):
        for line in txt.read_text().splitlines():
            utt, *rest = line.split(maxsplit=1)
            refs[utt] = rest[0] if rest else ""
        for ext in ("*.flac", "*.wav"):
            for wav in txt.parent.glob(ext):
                utt = wav.stem
                if utt in refs:
                    wavs.append(wav)
    wavs.sort()
    return wavs, refs

def main() -> None:
    model_dir = Path(__file__).parent / MODEL_DIR
    corpus_dir = Path(__file__).parent / "corpora/LibriSpeech/test-clean"
    wavs, refs = scan_librispeech(corpus_dir)

    recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
        tokens      = str(model_dir / "tokens.txt"),
        encoder     = str(model_dir / "encoder.onnx"),
        decoder     = str(model_dir / "decoder.onnx"),
        joiner      = str(model_dir / "joiner.onnx"),
        num_threads = 6,
        provider    = "cuda",
    )

    total_audio, total_decode = 0.0, 0.0
    hyps: List[str] = []

    for wav in tqdm(wavs, unit="utt"):
        audio, sr = sf.read(wav, dtype="float32")
        frame_size = int(sr * FRAME_DUR)
        stream = recognizer.create_stream()
        decode_time = 0.0

        for offset in range(0, len(audio), frame_size):
            chunk = audio[offset : offset + frame_size]
            stream.accept_waveform(sr, chunk)

            while recognizer.is_ready(stream):
                t0 = time.perf_counter()
```

```
                recognizer.decode_stream(stream)
                t1 = time.perf_counter()
                decode_time += (t1 - t0)

        while recognizer.is_ready(stream):
            t0 = time.perf_counter()
            recognizer.decode_stream(stream)
            t1 = time.perf_counter()
            decode_time += (t1 - t0)

        hyp = recognizer.get_result(stream)
        hyps.append(hyp)

        audio_len = len(audio) / sr
        total_audio += audio_len
        total_decode += decode_time

    print("\n=== LibriSpeech summary ===")
    print(f"Total utterances: {len(wavs)}")
    print(f"WER : {wer([refs[w.stem] for w in wavs], hyps):.4f}")
    print(f"RTF : {total_decode / total_audio:.4f}")

if __name__ == "__main__":
    main()
```

A.4 sherpa_service.py Script

```
import time
import threading
import numpy as np
from queue import Queue, Empty
from pathlib import Path
from scipy.signal import resample
import sherpa_onnx
import traceback
from sic_framework import SICComponentManager, SICConfMessage
from sic_framework.core.component_python2 import SICComponent
from sic_framework.core.connector import SICConnector
from sic_framework.core.message_python2 import AudioMessage, SICMessage, SICRequest

class SherpaConf(SICConfMessage):
    def __init__(self):
        super(SICConfMessage, self).__init__()
        model_dir = Path(__file__).parent / "sherpa-onnx-streaming-
        zipformer-en-2023-06-21"
        self.tokens  = str(model_dir / "tokens.txt")
```

```
        self.encoder = str(model_dir / "encoder.onnx")
        self.decoder = str(model_dir / "decoder.onnx")
        self.joiner  = str(model_dir / "joiner.onnx")

class GetTranscript(SICRequest):
    pass

class Transcript(SICMessage):
    def __init__(self, transcript: str):
        super().__init__()
        self.transcript = transcript

class SherpaASRWrapper:
    def __init__(self, conf: SherpaConf):
        for f in (conf.tokens, conf.encoder, conf.decoder, conf.joiner):
            assert Path(f).is_file(), f"Model file not found: {f}"

        self.recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
            tokens      = conf.tokens,
            encoder     = conf.encoder,
            decoder     = conf.decoder,
            joiner      = conf.joiner,
            num_threads = 6,
            provider    = "cuda",
        )
        self.stream      = self.recognizer.create_stream()
        self.audio_queue = Queue()
        self.last_result = ""
        self.running     = True
        self.thread = threading.Thread(target=self._decode_loop, daemon=True)
        self.thread.start()

    def accept_audio(self, samples: np.ndarray, sample_rate: int):
        self.audio_queue.put((samples, sample_rate))

    def _decode_loop(self):
        while self.running:
            try:
                samples, sample_rate = self.audio_queue.get(timeout=0.1)
                self.stream.accept_waveform(sample_rate, samples)

                while self.recognizer.is_ready(self.stream):
                    self.recognizer.decode_stream(self.stream)

                text = self.recognizer.get_result(self.stream)
                if text and text != self.last_result:
```

```
                    self.last_result = text
                    self.on_new_transcript(text)

            except Empty:
                continue
            except Exception:
                traceback.print_exc()

    def stop(self):
        self.running = False
        self.thread.join()

    def on_new_transcript(self, text: str):
        pass

class SherpaComponent(SICComponent):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.wrapper = SherpaASRWrapper(self.params)
        self.wrapper.on_new_transcript = self._publish_transcript

    @staticmethod
    def get_inputs():
        return [AudioMessage, GetTranscript]

    @staticmethod
    def get_output():
        return Transcript

    @staticmethod
    def get_conf():
        return SherpaConf()

    def on_message(self, message: AudioMessage):
        audio = np.frombuffer(message.waveform, dtype=np.int16).astype(np.float32) /
        32768.0
        if message.sample_rate != 16000:
            target_len = int(len(audio) * 16000 / message.sample_rate)
            audio = resample(audio, target_len)
            sr = 16000
        else:
            sr = message.sample_rate
        self.wrapper.accept_audio(audio, sr)

    def on_request(self, request: SICRequest):
        if isinstance(request, GetTranscript):
```

```
            return Transcript(self.wrapper.last_result or "")
        return Transcript("")

    def _publish_transcript(self, text: str):
        self.output_message(Transcript(text))

class SICSherpa(SICConnector):
    component_class = SherpaComponent

def main():
    SICComponentManager([SherpaComponent])

if __name__ == "__main__":
    main()
```

A.5 demo_desktop_microphone_sherpa.py Script

```
import time
from sic_framework.devices.desktop import Desktop
from sic_framework.services.sherpa_asr.sherpa_service import
SICSherpa, Transcript

def on_transcript(msg: Transcript):
    print("Sherpa Transcript:", msg.transcript)

desktop = Desktop()
sherpa  = SICSherpa()
sherpa.connect(desktop.mic)
sherpa.register_callback(on_transcript)

print("Start speaking! Ctrl+C to quit")
try:
    while True:
        time.sleep(0.2)
except KeyboardInterrupt:
    pass
finally:
    sherpa.stop()
```